

Fast National University of Computing and Emerging Sciences



Smart Disaster Response MIS

DATABASE SYSTEMS

Muhammad Jibran (24i-3134)

Alishba Nasir (24i-3176)

Ashar Ahmed (24i-3072)

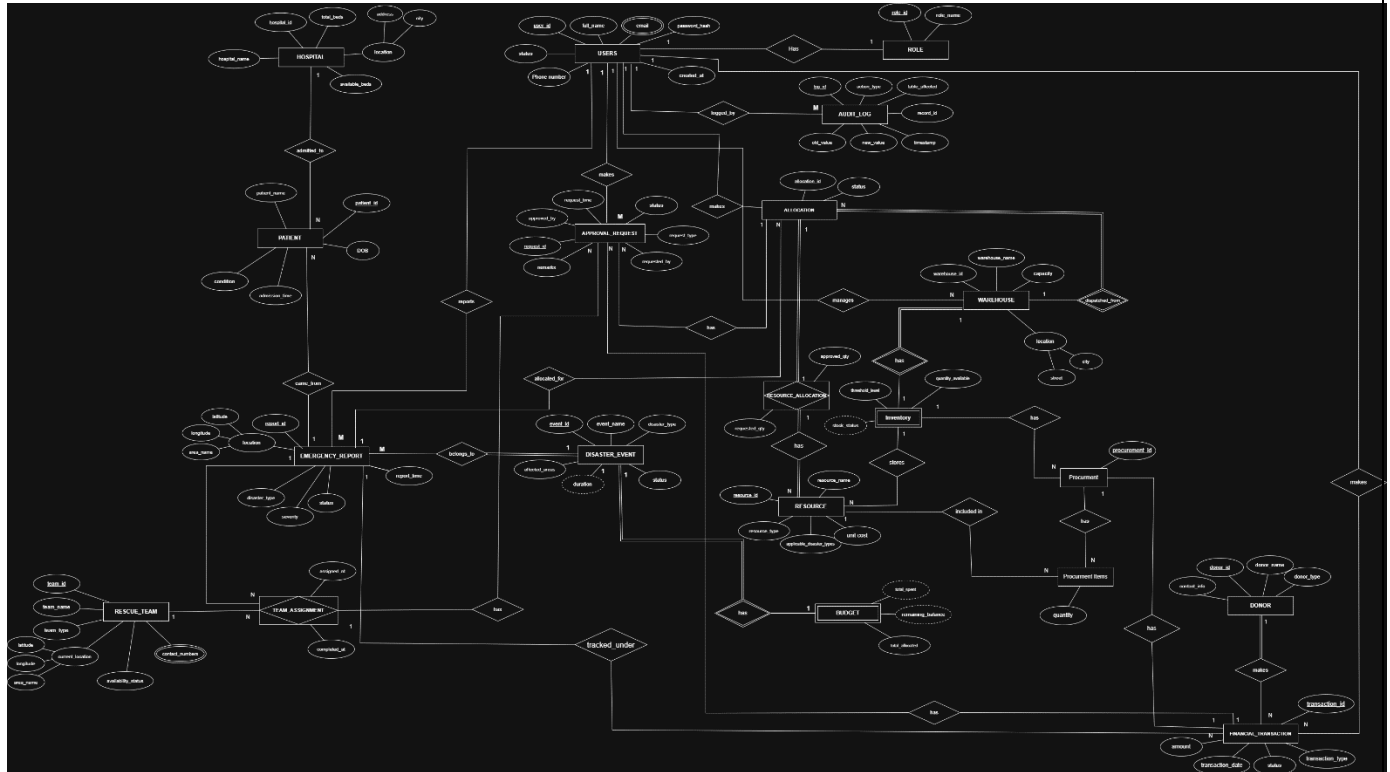
Submitted to: Miss Zoya Sumbul Zaheer

Table of Contents

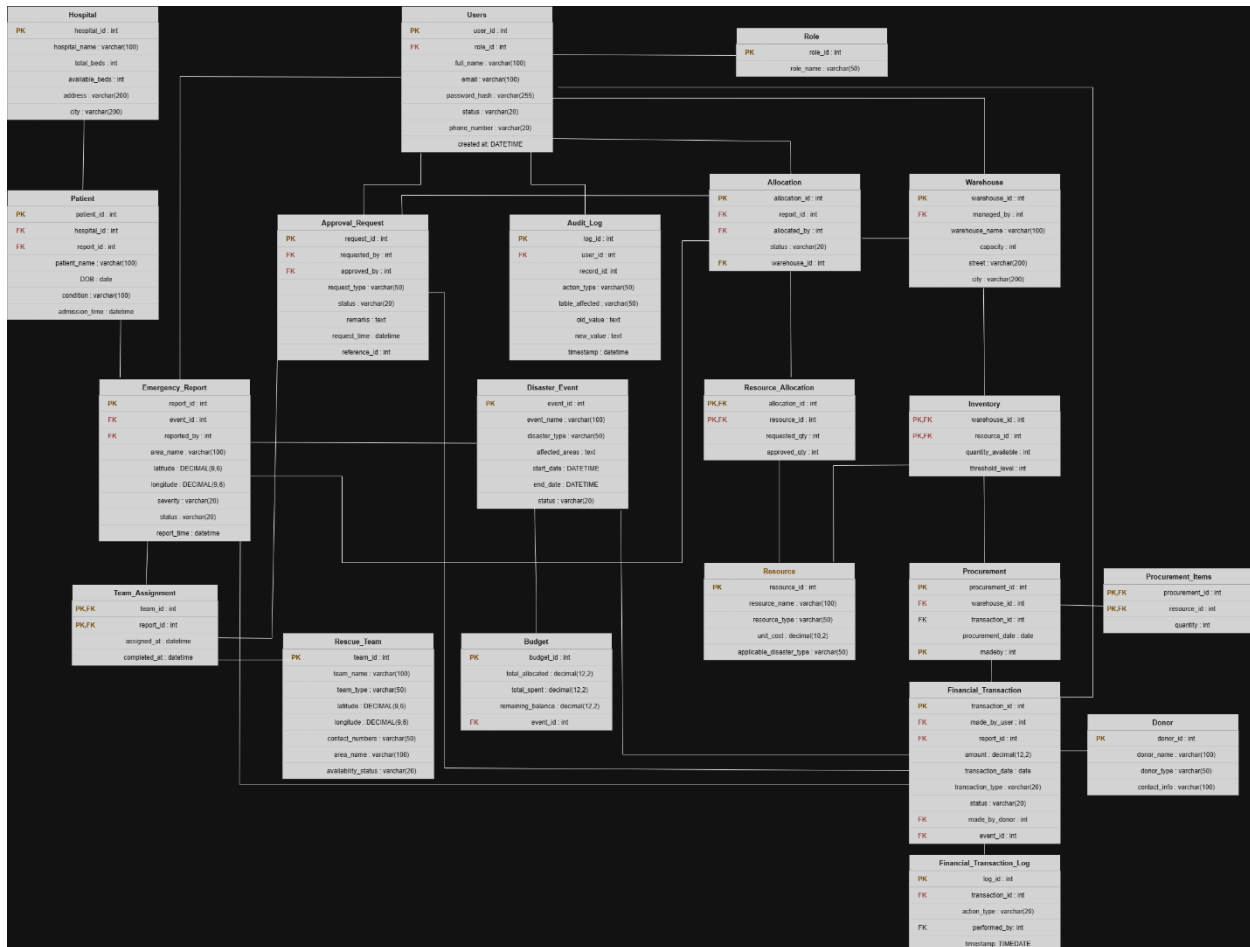
.....	1
EER Diagram.....	4
Relational Schema	5
Normalization	5
First Normal Form (1NF).....	5
Problems Identified.....	5
Changes Applied to Achieve 1NF	5
Second Normal Form (2NF)	6
Partial Dependencies Identified	6
Third Normal Form (3NF).....	7
Transitive Dependencies Identified	7
Final Normalized Schema.....	8
SQL Implementation	8
DDL.....	8
DML.....	19
RBAC Implementation Strategy	22
View Definitions & Performance Comparison (Latency Analysis).....	23
Introduction.....	23
Implemented Views	23
finance_view	23
field_officer_view.....	23
warehouse_view.....	23
admin_view.....	23
Data Security and Access Control	24
Performance and Latency Evaluation	24
Experimental Results	24
Test 1: Direct Table Query (Raw Joins)	24
Test 2: View-Based Query.....	24
Performance Comparison.....	24
Analysis	24
Faster Response	24
Better Security and Abstraction	24

Simplified Querying.....	24
Performance Optimization & Indexing Strategy	25
Introduction.....	25
Custom Indexing.....	25
Single-Column Indexes.....	25
Composite Indexes.....	25
Query Performance Analysis	25
without Index (Full Table Scan)	25
with Index	26
Index Overhead (Insert Operation)	26
Performance Comparison.....	26
Key Findings.....	26
Where Indexing Helps	26
Where Indexing Adds Cost.....	26
Without index.....	27
With index.....	27
Index overhead.....	27
Frontend Interface.....	28
Database Design Decisions	31
Entity Choices and Relationships	32
View Abstraction and Security	32
MIS Reports / Dashboards	33
Github Repo link.....	33

EER Diagram



Relational Schema



Normalization

First Normal Form (1NF)

A table is in First Normal Form when every attribute contains only atomic values, every column contains values of the same type, and each row is uniquely identifiable.

Problems Identified

The following violations of 1NF were found in the unnormalized design:

- Multi-valued attribute _ Rescue Team Contact Numbers: A single cell stored multiple phone numbers (e.g., "0300-1234567, 0321-9876543"). This violates atomicity.

Changes Applied to Achieve 1NF

- The multi-valued contact number column was extracted into a separate Team_Contact table, where each phone number occupies its own row, linked back to the Rescue_Team via a foreign key.

Second Normal Form (2NF)

A table is in Second Normal Form when it is already in 1NF and every non-key attribute is fully functionally dependent on the entire primary key (no partial dependencies). Partial dependencies only arise in tables with composite primary keys.

Partial Dependencies Identified

1. Inventory Table : (warehouse_id, resource_id)

After 1NF, the Inventory table had the composite primary key (warehouse_id, resource_id). The following partial dependencies were detected:

warehouse_id : warehouse_name, city, capacity, managed_by **resource_id** : resource_name, resource_type, unit_cost

The attributes warehouse_name, city, capacity, and managed_by depend only on warehouse_id, not on the full composite key. Similarly, resource_name, resource_type, and unit_cost depend only on resource_id.

Resolution: These attributes were moved to dedicated Warehouse and Resource tables respectively. The Inventory table retains only (warehouse_id, resource_id, quantity_available, threshold_level), values that genuinely depend on the combination of both keys.

2. Resource_Allocation Table : (allocation_id, resource_id)

The composite key (allocation_id, resource_id) had a partial dependency:

allocation_id : report_id, allocated_by, warehouse_id, status

Attributes such as report_id, allocated_by, warehouse_id, and status depend solely on allocation_id. Only requested_qty and approved_qty genuinely depend on both allocation_id and resource_id.

Resolution: Allocation-level attributes were moved to a parent Allocation table. Resource_Allocation retains only (allocation_id, resource_id, requested_qty, approved_qty).

3. Team_Assignment Table : (team_id, report_id)

A partial dependency was present:

team_id : team_name, team_type, latitude, longitude, availability_status **report_id** : area_name, severity, event_id, reported_by

Resolution: All team attributes were placed in Rescue_Team and all report attributes in Emergency_Report. Team_Assignment retains only (team_id, report_id, assigned_at, completed_at).

4. Procurement_Items Table : (procurement_id, resource_id)

Partial dependencies found:

procurement_id : procured_by, warehouse_id, transaction_id, procurement_date **resource_id** :
resource_name, resource_type, unit_cost

Resolution: Procurement-level attributes moved to Procurement table; resource attributes to Resource table. Procurement_Items retains only (procurement_id, resource_id, quantity).

Third Normal Form (3NF)

A table is in Third Normal Form when it is already in 2NF and every non-key attribute is nontransitively dependent on the primary key that is, no non-key attribute determines another non-key attribute.

Transitive Dependencies Identified

1. Users : Role Attributes

In an earlier design, the Users table contained role_name and role_permissions directly:

user_id : role_name, role_permissions, full_name, email **role_name** :
role_permissions

This creates a transitive dependency: user_id : role_name = role_permissions.

Resolution: A separate Role table was created with role_id as the primary key and role_name as a unique attribute. Users now stores only the foreign key role_id, removing the transitive dependency. Role names are constrained via a CHECK constraint to the five permitted system roles.

2. Emergency_Report : Event Attributes

In the flat design, Emergency_Report included event-level attributes:

report_id : event_name, disaster_type, event_status, start_date **event_id** :
event_name, disaster_type, event_status, start_date

Transitive path: report_id = event_id = event attributes.

Resolution: All event-level attributes were moved to the Disaster_Event table. Emergency_Report stores only the foreign key event_id, eliminating the transitive dependency.

3. *Financial_Transaction : Donor Attributes*

A Financial_Transaction table contained donor attributes inline:

transaction_id = donor_name, donor_type, contact_info, amount **donor_id** = donor_name, donor_type, contact_info

Transitive path: transaction_id = donor_id = donor attributes.

Resolution: Donor was extracted into its own table. Financial_Transaction now references donors via the foreign key made_by_donor.

Final Normalized Schema

After applying 1NF, 2NF, and 3NF, the final schema consists of 20 tables.

SQL Implementation

DDL

```
CREATE DATABASE DisasterResponseDB

USE DisasterResponseDB

CREATE TABLE Role (
    role_id INT IDENTITY(1,1) PRIMARY KEY,
    role_name VARCHAR(50) NOT NULL UNIQUE,
    CONSTRAINT chk_role_name CHECK (
        role_name IN ('Administrator','Emergency Operator','Field Officer',
            'Warehouse Manager','Finance Officer')
    )
);

CREATE TABLE Users (
    user_id INT IDENTITY(1,1) PRIMARY KEY,
    role_id INT NOT NULL,
    full_name VARCHAR(100) NOT NULL,
    email VARCHAR(100) NOT NULL UNIQUE,
    password_hash VARCHAR(255) NOT NULL,
    status VARCHAR(20) NOT NULL DEFAULT 'Active',
    phone_number VARCHAR(20),
    created_at DATETIME DEFAULT GETDATE(),

    CONSTRAINT chk_user_status CHECK (status IN
('Active','Inactive','Suspended')),
    CONSTRAINT fk_user_role FOREIGN KEY (role_id) REFERENCES Role(role_id)
);
```

```

CREATE TABLE Hospital (
    hospital_id INT IDENTITY(1,1) PRIMARY KEY,
    hospital_name VARCHAR(100) NOT NULL,
    total_beds INT NOT NULL,
    available_beds INT NOT NULL,
    address VARCHAR(200) NOT NULL,
    city VARCHAR(200) NOT NULL,

    CONSTRAINT chk_beds CHECK (available_beds >= 0 AND available_beds <=
total_beds)
);

CREATE TABLE Disaster_Event (
    event_id INT IDENTITY(1,1) PRIMARY KEY,
    event_name VARCHAR(100) NOT NULL,
    disaster_type VARCHAR(50) NOT NULL,
    affected_areas VARCHAR(MAX),
    start_date DATETIME NOT NULL,
    end_date DATETIME NULL,
    status VARCHAR(20) NOT NULL DEFAULT 'Active',

    CONSTRAINT chk_disaster_type CHECK (
        disaster_type IN
('Flood', 'Earthquake', 'Fire', 'Cyclone', 'Landslide', 'Other')
    ),
    CONSTRAINT chk_event_status CHECK (
        status IN ('Active', 'Contained', 'Resolved', 'Closed')
    ),
    CONSTRAINT chk_event_dates CHECK (end_date IS NULL OR end_date >= start_date)
);

CREATE TABLE Emergency_Report (
    report_id INT IDENTITY(1,1) PRIMARY KEY,
    event_id INT NOT NULL,
    reported_by INT NOT NULL,
    area_name VARCHAR(100) NOT NULL,
    latitude DECIMAL(9,6) NOT NULL,
    longitude DECIMAL(9,6) NOT NULL,
    severity VARCHAR(20) NOT NULL,
    status VARCHAR(20) NOT NULL DEFAULT 'Open',
    report_time DATETIME DEFAULT GETDATE(),

    CONSTRAINT chk_severity CHECK (severity IN
('Low', 'Medium', 'High', 'Critical')),

```

```

        CONSTRAINT chk_report_status CHECK (status IN ('Open','In
Progress','Resolved','Closed')),

        CONSTRAINT fk_report_event FOREIGN KEY (event_id) REFERENCES
Disaster_Event(event_id),
        CONSTRAINT fk_report_user FOREIGN KEY (reported_by) REFERENCES
[Users](user_id)
    );

CREATE TABLE Patient (
    patient_id INT IDENTITY(1,1) PRIMARY KEY,
    hospital_id INT NOT NULL,
    report_id INT NULL,
    patient_name VARCHAR(100) NOT NULL,
    dob DATE,
    condition VARCHAR(100),
    admission_time DATETIME DEFAULT GETDATE(),

    CONSTRAINT fk_patient_hospital FOREIGN KEY (hospital_id) REFERENCES
Hospital(hospital_id),
    CONSTRAINT fk_patient_report FOREIGN KEY (report_id) REFERENCES
Emergency_Report(report_id)
);

CREATE TABLE Rescue_Team (
    team_id INT IDENTITY(1,1) PRIMARY KEY,
    team_name VARCHAR(100) NOT NULL,
    team_type VARCHAR(50) NOT NULL,
    latitude DECIMAL(9,6),
    longitude DECIMAL(9,6),
    area_name VARCHAR(100),
    availability_status VARCHAR(20) NOT NULL DEFAULT 'Available',

    CONSTRAINT chk_team_type CHECK (
        team_type IN ('Medical','Fire','Rescue','Search','Logistics')
    ),
    CONSTRAINT chk_team_status CHECK (
        availability_status IN ('Available','Assigned','Busy','Completed')
    )
);

CREATE TABLE Team_Contact (
    contact_id INT IDENTITY(1,1) PRIMARY KEY,
    team_id INT NOT NULL,
    contact_number VARCHAR(20) NOT NULL,
    FOREIGN KEY (team_id) REFERENCES Rescue_Team(team_id)
);

```

```

        ON DELETE CASCADE
    );

CREATE TABLE Approval_Request (
    request_id INT IDENTITY(1,1) PRIMARY KEY,
    requested_by INT NOT NULL,
    approved_by INT NULL,
    reference_id INT NULL,
    request_type VARCHAR(50) NOT NULL,
    status VARCHAR(20) NOT NULL DEFAULT 'Pending',
    remarks VARCHAR(MAX),
    request_time DATETIME DEFAULT GETDATE(),

    CONSTRAINT chk_approval_status CHECK (
        status IN ('Pending', 'Approved', 'Rejected', 'Cancelled')
    ),

    CONSTRAINT fk_req_user FOREIGN KEY (requested_by) REFERENCES
[Users](user_id),
    CONSTRAINT fk_app_user FOREIGN KEY (approved_by) REFERENCES [Users](user_id)
);

CREATE TABLE Team_Assignment (
    team_id INT NOT NULL,
    report_id INT NOT NULL,
    assigned_at DATETIME DEFAULT GETDATE(),
    completed_at DATETIME NULL,

    PRIMARY KEY (team_id, report_id),

    CONSTRAINT fk_team FOREIGN KEY (team_id) REFERENCES Rescue_Team(team_id),
    CONSTRAINT fk_report FOREIGN KEY (report_id) REFERENCES
Emergency_Report(report_id)
);

CREATE TABLE Warehouse (
    warehouse_id INT IDENTITY(1,1) PRIMARY KEY,
    managed_by INT NOT NULL,
    warehouse_name VARCHAR(100) NOT NULL,
    capacity INT NOT NULL,
    street VARCHAR(200),
    city VARCHAR(200) NOT NULL,

    CONSTRAINT fk_wh_user FOREIGN KEY (managed_by) REFERENCES [Users](user_id)
);

```

```

CREATE TABLE Resource (
    resource_id INT IDENTITY(1,1) PRIMARY KEY,
    resource_name VARCHAR(100) NOT NULL,
    resource_type VARCHAR(50) NOT NULL,
    unit_cost DECIMAL(10,2) NOT NULL,

    CONSTRAINT chk_resource_type CHECK (
        resource_type IN
('Food','Water','Medicine','Shelter','Equipment','Vehicle','Other')
    )
);

CREATE TABLE Inventory (
    warehouse_id INT NOT NULL,
    resource_id INT NOT NULL,
    quantity_available INT DEFAULT 0,
    threshold_level INT DEFAULT 10,

    PRIMARY KEY (warehouse_id, resource_id),

    CONSTRAINT fk_inv_wh FOREIGN KEY (warehouse_id) REFERENCES
Warehouse(warehouse_id),
    CONSTRAINT fk_inv_res FOREIGN KEY (resource_id) REFERENCES
Resource(resource_id)
);

CREATE TABLE Allocation (
    allocation_id INT IDENTITY(1,1) PRIMARY KEY,
    report_id INT NOT NULL,
    allocated_by INT NOT NULL,
    warehouse_id INT NULL,
    status VARCHAR(20) DEFAULT 'Pending',

    CONSTRAINT chk_alloc_status CHECK (
        status IN ('Pending','Approved','Dispatched','Delivered','Cancelled')
    ),

    CONSTRAINT fk_alloc_report FOREIGN KEY (report_id) REFERENCES
Emergency_Report(report_id),
    CONSTRAINT fk_alloc_user FOREIGN KEY (allocated_by) REFERENCES
[Users](user_id),
    CONSTRAINT fk_alloc_wh FOREIGN KEY (warehouse_id) REFERENCES
Warehouse(warehouse_id)
);

```

```

CREATE TABLE Resource_Allocation (
    allocation_id INT NOT NULL,
    resource_id INT NOT NULL,
    requested_qty INT NOT NULL,
    approved_qty INT NULL,

    PRIMARY KEY (allocation_id, resource_id),

    CONSTRAINT fk_ra_alloc FOREIGN KEY (allocation_id) REFERENCES
Allocation(allocation_id),
    CONSTRAINT fk_ra_res FOREIGN KEY (resource_id) REFERENCES
Resource(resource_id)
);

CREATE TABLE Donor (
    donor_id INT IDENTITY(1,1) PRIMARY KEY,
    donor_name VARCHAR(100) NOT NULL,
    donor_type VARCHAR(50),
    contact_info VARCHAR(100)
);

CREATE TABLE Financial_Transaction (
    transaction_id INT IDENTITY(1,1) PRIMARY KEY,

    made_by_user INT NULL,
    made_by_donor INT NULL,
    event_id INT NULL,
    amount DECIMAL(12,2) NOT NULL,
    transaction_date DATETIME DEFAULT GETDATE(),
    transaction_type VARCHAR(20) NOT NULL,
    status VARCHAR(20) DEFAULT 'Pending',
    CONSTRAINT chk_ft_type CHECK (transaction_type IN
('Donation', 'Expense', 'Procurement', 'Transfer')),
    CONSTRAINT chk_ft_status CHECK (status IN
('Pending', 'Completed', 'Failed', 'Reversed')),
    CONSTRAINT chk_ft_actor CHECK ((made_by_user IS NOT NULL AND made_by_donor IS
NULL) OR
    (made_by_user IS NULL AND made_by_donor IS NOT NULL)
),

    CONSTRAINT fk_ft_user FOREIGN KEY (made_by_user) REFERENCES [Users](user_id),
    CONSTRAINT fk_ft_donor FOREIGN KEY (made_by_donor) REFERENCES
Donor(donor_id),

```

```

        CONSTRAINT fk_ft_event FOREIGN KEY (event_id) REFERENCES
Disaster_Event(event_id)
);
CREATE TABLE Procurement (
    procurement_id INT IDENTITY(1,1) PRIMARY KEY,
    procured_by INT NOT NULL,
    warehouse_id INT NOT NULL,
    transaction_id INT NULL,
    procurement_date DATETIME DEFAULT GETDATE(),

    CONSTRAINT fk_proc_user FOREIGN KEY (procured_by) REFERENCES
[Users](user_id),
    CONSTRAINT fk_proc_wh FOREIGN KEY (warehouse_id) REFERENCES
Warehouse(warehouse_id),
    CONSTRAINT fk_proc_ft FOREIGN KEY (transaction_id) REFERENCES
Financial_Transaction(transaction_id)
);

CREATE TABLE Procurement_Items (
    procurement_id INT NOT NULL,
    resource_id INT NOT NULL,
    quantity INT NOT NULL,

    PRIMARY KEY (procurement_id, resource_id),

    CONSTRAINT fk_pi_proc FOREIGN KEY (procurement_id) REFERENCES
Procurement(procurement_id),
    CONSTRAINT fk_pi_res FOREIGN KEY (resource_id) REFERENCES
Resource(resource_id)
);

CREATE TABLE Budget (
    budget_id INT IDENTITY(1,1) PRIMARY KEY,
    event_id INT NOT NULL,
    total_allocated DECIMAL(12,2) NOT NULL,
    total_spent DECIMAL(12,2) NOT NULL DEFAULT 0,
    remaining_balance AS (total_allocated - total_spent),

    CONSTRAINT fk_budget_event FOREIGN KEY (event_id) REFERENCES
Disaster_Event(event_id)
);

CREATE TABLE Financial_Transaction_Log (
    log_id INT IDENTITY(1,1) PRIMARY KEY,
    transaction_id INT NOT NULL,

```

```

    action_type VARCHAR(20) NOT NULL,
    performed_by INT NOT NULL,
    log_time DATETIME DEFAULT GETDATE(),

    CONSTRAINT fk_ftl_trans FOREIGN KEY (transaction_id) REFERENCES
Financial_Transaction(transaction_id),
    CONSTRAINT fk_ftl_user FOREIGN KEY (performed_by) REFERENCES [Users](user_id)
);

CREATE TABLE Audit_Log (
    log_id INT IDENTITY(1,1) PRIMARY KEY,
    user_id INT NOT NULL,
    record_id INT NULL,
    action_type VARCHAR(50),
    table_name VARCHAR(50),
    old_value VARCHAR(MAX),
    new_value VARCHAR(MAX),
    log_time DATETIME DEFAULT GETDATE(),

    CONSTRAINT fk_audit_user FOREIGN KEY (user_id) REFERENCES [Users](user_id)
);

-- STORED PROCEDURES – ACID TRANSACTIONS
-- All multi-step operations use BEGIN TRAN / COMMIT / ROLLBACK inside

GO

-- SP 1: sp_AllocateResource
-- Steps: Check stock -> Create Allocation -> Create detail row Deduct inventory
CREATE OR ALTER PROCEDURE sp_AllocateResource
    @report_id      INT,
    @allocated_by   INT,
    @warehouse_id   INT,
    @resource_id     INT,
    @quantity        INT,
    @allocation_id  INT          OUTPUT,
    @remaining_stock INT          OUTPUT,
    @error_message   VARCHAR(500) OUTPUT
AS
BEGIN
    SET NOCOUNT ON;
    SET @allocation_id = NULL;
    SET @remaining_stock = NULL;

```

```

SET @error_message = NULL;

BEGIN TRY
    BEGIN TRANSACTION;

    DECLARE @current_stock INT;
    SELECT @current_stock = quantity_available
    FROM Inventory
    WHERE warehouse_id = @warehouse_id AND resource_id = @resource_id;

    IF @current_stock IS NULL
        RAISERROR('No inventory record found for warehouse %d, resource %d.',
16, 1, @warehouse_id, @resource_id);

    IF @current_stock < @quantity
        RAISERROR('Not enough stock. Available: %d, Requested: %d.', 16, 1,
@current_stock, @quantity);

    INSERT INTO Allocation (report_id, allocated_by, warehouse_id, status)
    VALUES (@report_id, @allocated_by, @warehouse_id, 'Dispatched');

    SET @allocation_id = SCOPE_IDENTITY();

    INSERT INTO Resource_Allocation (allocation_id, resource_id,
requested_qty, approved_qty)
    VALUES (@allocation_id, @resource_id, @quantity, @quantity);

    UPDATE Inventory
    SET quantity_available = quantity_available - @quantity
    WHERE warehouse_id = @warehouse_id AND resource_id = @resource_id;

    SET @remaining_stock = @current_stock - @quantity;

    COMMIT TRANSACTION;
END TRY
BEGIN CATCH
    IF @@TRANCOUNT > 0 ROLLBACK TRANSACTION;
    SET @error_message = ERROR_MESSAGE();
END CATCH
END;
GO

-- SP 2: sp_AssignTeam

```

-- Steps: Check availability -> Insert assignment -> Mark team Assigned -> Report In Progress

```
CREATE OR ALTER PROCEDURE sp_AssignTeam
    @team_id      INT,
    @report_id    INT,
    @error_message VARCHAR(500) OUTPUT
AS
BEGIN
    SET NOCOUNT ON;
    SET @error_message = NULL;

    BEGIN TRY
        BEGIN TRANSACTION;

        DECLARE @team_status VARCHAR(20);
        SELECT @team_status = availability_status
        FROM Rescue_Team WHERE team_id = @team_id;

        IF @team_status IS NULL
            RAISERROR('Team with id %d not found.', 16, 1, @team_id);

        IF @team_status <> 'Available'
            RAISERROR('Team is not available. Current status: %s.', 16, 1,
@team_status);

        INSERT INTO Team_Assignment (team_id, report_id, assigned_at)
        VALUES (@team_id, @report_id, GETDATE());

        UPDATE Rescue_Team SET availability_status = 'Assigned' WHERE team_id =
@team_id;

        UPDATE Emergency_Report SET status = 'In Progress' WHERE report_id =
@report_id;

        COMMIT TRANSACTION;
    END TRY
    BEGIN CATCH
        IF @@TRANCOUNT > 0 ROLLBACK TRANSACTION;
        SET @error_message = ERROR_MESSAGE();
    END CATCH
END;
GO
```

```

-- SP 3: sp_FinancialEntry
-- Steps: Insert transaction -> Audit log -> Update budget (if Expense)

CREATE OR ALTER PROCEDURE sp_FinancialEntry
    @made_by_user      INT          = NULL,
    @made_by_donor     INT          = NULL,
    @event_id          INT          = NULL,
    @amount            DECIMAL(12,2),
    @transaction_type  VARCHAR(20),
    @performed_by      INT,
    @transaction_id    INT          OUTPUT,
    @error_message     VARCHAR(500) OUTPUT
AS
BEGIN
    SET NOCOUNT ON;
    SET @transaction_id = NULL;
    SET @error_message = NULL;

    IF (@made_by_user IS NULL AND @made_by_donor IS NULL)
        OR (@made_by_user IS NOT NULL AND @made_by_donor IS NOT NULL)
    BEGIN
        SET @error_message = 'Provide either @made_by_user OR @made_by_donor (not both).';
        RETURN;
    END

    BEGIN TRY
        BEGIN TRANSACTION;

        INSERT INTO Financial_Transaction
            (made_by_user, made_by_donor, event_id, amount, transaction_type,
status)
        VALUES
            (@made_by_user, @made_by_donor, @event_id, @amount,
@transaction_type, 'Completed');

        SET @transaction_id = SCOPE_IDENTITY();

        DECLARE @new_value VARCHAR(MAX);
        SET @new_value = @transaction_type + ' of ' + CAST(@amount AS
VARCHAR(20))
            + ' | txn_id=' + CAST(@transaction_id AS VARCHAR(10));

        INSERT INTO Audit_Log (user_id, record_id, action_type, table_name,
old_value, new_value)

```

```

        VALUES (@performed_by, @transaction_id, 'INSERT',
'Financial_Transaction', NULL, @new_value);

        IF @transaction_type = 'Expense' AND @event_id IS NOT NULL
        BEGIN
            UPDATE Budget SET total_spent = total_spent + @amount WHERE event_id
= @event_id;
        END

        COMMIT TRANSACTION;
    END TRY
    BEGIN CATCH
        IF @@TRANCOUNT > 0 ROLLBACK TRANSACTION;
        SET @error_message = ERROR_MESSAGE();
    END CATCH
END;
GO

```

DML

```

USE DisasterResponseDB;

INSERT INTO Role (role_name) VALUES
('Administrator'),
('Emergency Operator'),
('Field Officer'),
('Warehouse Manager'),
('Finance Officer');

INSERT INTO Users (role_id, full_name, email, password_hash, status,
phone_number) VALUES
(1, 'Jibran', 'i243134@isb.nu.edu.pk', '$2b$12$KIX8...adminHash', 'Active',
'0345-1234567'),
(2, 'Alishba', 'i243176@isb.nu.edu.pk', '$2b$12$KIX8...adminHash', 'Active',
'0345-1234567'),
(3, 'Ashar', 'i243072@isb.nu.edu.pk', '$2b$12$KIX8...adminHash', 'Active',
'0345-1234567');

INSERT INTO Hospital (hospital_name, total_beds, available_beds, address, city)
VALUES
('Jinnah Hospital', 500, 120, 'Shad Bagh Road', 'Lahore'),
('PIMS Hospital', 600, 200, 'G-8/3', 'Islamabad'),

```

```

('Civil Hospital Karachi', 800, 300, 'Karachi Cantt', 'Karachi');

INSERT INTO Disaster_Event (event_name, disaster_type, affected_areas,
start_date, end_date, status) VALUES
('Punjab Flood', 'Flood', 'DG Khan, Muzaffargarh', '2024-07-15', NULL, 'Active'),
('KPK Earthquake', 'Earthquake', 'Swat, Dir', '2024-10-05', '2024-10-05',
'Resolved'),
('Karachi Fire', 'Fire', 'SITE Area', '2024-11-20', '2024-11-21', 'Resolved');

INSERT INTO Emergency_Report (event_id, reported_by, area_name, latitude,
longitude, severity, status, report_time) VALUES
(1, 2, 'DG Khan City', 29.96, 70.63, 'Critical', 'In Progress', '2024-07-15'),
(1, 3, 'Muzaffargarh', 30.07, 71.18, 'High', 'Open', '2024-07-15'),
(2, 2, 'Swat', 34.77, 72.36, 'Critical', 'Resolved', '2024-10-05'),
(2, 3, 'Dir', 35.26, 71.84, 'Medium', 'Resolved', '2024-10-05'),
(3, 2, 'SITE Area', 24.89, 67.01, 'High', 'Closed', '2024-11-20');

INSERT INTO Patient (hospital_id, report_id, patient_name, dob, condition,
admission_time) VALUES
(1, 1, 'Ali Khan', '1980-05-10', 'Drowning', '2024-07-15'),
(1, 2, 'Sara Bibi', '1995-03-20', 'Injury', '2024-07-15'),
(2, 3, 'Usman Ali', '1975-08-12', 'Fracture', '2024-10-05'),
(3, 5, 'Ahmed Raza', '1990-11-01', 'Burns', '2024-11-20');

INSERT INTO Rescue_Team (team_name, team_type, latitude, longitude, area_name,
availability_status) VALUES
('Alpha Medical', 'Medical', 31.52, 74.35, 'Lahore', 'Available'),
('Bravo Fire', 'Fire', 33.72, 73.04, 'Islamabad', 'Available'),
('Charlie Search', 'Search', 24.86, 67.00, 'Karachi', 'Assigned'),
('Delta Rescue', 'Rescue', 34.01, 71.52, 'Peshawar', 'Available'),
('Echo Logistics', 'Logistics', 30.15, 71.52, 'Multan', 'Available');

INSERT INTO Team_Contact (team_id, contact_number)
VALUES
(1, '03001234567'),
(1, '03111234567');

INSERT INTO Warehouse (managed_by, warehouse_name, capacity, street, city) VALUES
(1, 'Lahore Depot', 5000, 'Ferozepur Road', 'Lahore'),
(1, 'Islamabad Store', 4000, 'H-9', 'Islamabad'),
(2, 'Karachi Warehouse', 8000, 'West Wharf', 'Karachi'),
(3, 'Quetta Store', 2500, 'Brewery Road', 'Quetta');

INSERT INTO Resource (resource_name, resource_type, unit_cost) VALUES
('Food Pack', 'Food', 500),

```

```

('Water Bottle', 'Water', 100),
('First Aid Kit', 'Medicine', 2000),
('Tent', 'Shelter', 15000),
('Fire Extinguisher', 'Equipment', 3000),
('Ambulance', 'Vehicle', 40000);

INSERT INTO Inventory (warehouse_id, resource_id, quantity_available,
threshold_level) VALUES
(1, 1, 200, 50),
(1, 2, 150, 30),
(2, 3, 80, 20),
(2, 4, 40, 10),
(3, 5, 60, 15),
(4, 1, 100, 25);

INSERT INTO Allocation (report_id, allocated_by, warehouse_id, status) VALUES
(1, 1, 1, 'Dispatched'),
(2, 1, 1, 'Pending'),
(3, 2, 2, 'Delivered'),
(5, 2, 3, 'Delivered');

INSERT INTO Resource_Allocation (allocation_id, resource_id, requested_qty,
approved_qty) VALUES
(1, 1, 100, 100),
(1, 2, 50, 50),
(2, 1, 80, NULL),
(3, 3, 30, 30),
(4, 5, 20, 20);

INSERT INTO Donor (donor_name, donor_type, contact_info) VALUES
('Edhi Foundation', 'NGO', '021-32211009'),
('USAID', 'International', 'usaid.pk@usaid.gov');

INSERT INTO Financial_Transaction (made_by_user, made_by_donor, event_id, amount,
transaction_date, transaction_type, status) VALUES
(1, NULL, 1, 2000000, '2024-07-16', 'Expense', 'Completed'),
(NULL, 1, 2, 5000000, '2024-07-17', 'Donation', 'Completed');

INSERT INTO Budget (event_id, total_allocated, total_spent) VALUES
(1, 20000000, 5000000),
(2, 15000000, 7000000),
(3, 10000000, 4000000);

-- Insert missing Team Assignments for Response Time Analytics

```

```

INSERT INTO Team_Assignment (team_id, report_id, assigned_at, completed_at)
VALUES
(1, 1, '2024-07-15 10:00:00', '2024-07-15 14:30:00'),
(2, 5, '2024-11-20 08:00:00', '2024-11-20 09:15:00'),
(3, 3, '2024-10-05 06:00:00', '2024-10-05 18:00:00');

-- Insert missing Approval Requests for Approval Workflow History
INSERT INTO Approval_Request (requested_by, approved_by, reference_id,
request_type, status, remarks, request_time) VALUES
(2, 1, 1, 'Resource Allocation', 'Approved', 'Urgent food packs needed', '2024-
07-15'),
(3, NULL, 2, 'Budget Increase', 'Pending', 'Need more funds for rescue', '2024-
10-06'),
(2, 1, 3, 'Team Deployment', 'Approved', 'Send Bravo team', '2024-11-20');

```

Transaction Handling Demonstration

The system treats certain operations as single logical units that must either fully succeed or fully fail. This is the ACID principle in practice. Three critical workflows were wrapped in explicit transactions. The first is resource allocation, when a resource distribution request is approved, the system must simultaneously create an Allocation record, insert a Resource_Allocation detail row, and decrement

Inventory.quantity_available. If the inventory update fails for any reason, the allocation record should not exist either. Wrapping these three steps in a BEGIN TRANSACTION with ROLLBACK on failure guarantees this.

The second is team deployment, approving a deployment request simultaneously inserts a Team_Assignment record, updates the team's availability_status to Assigned, and marks the Emergency_Report as In Progress. All three changes represent one real-world action, so they must be atomic. The third is financial recording, creating a Financial_Transaction and logging it to Financial_Transaction_Log happen together, and if the event is an expense, the relevant Budget.total_spent is incremented in the same transaction. Isolation is maintained through SQL Server's default transaction isolation level, which prevents dirty reads between concurrent operations.

RBAC Implementation Strategy

Five roles were defined based on the system requirements: Administrator, Emergency Operator, Field Officer, Warehouse Manager, and Finance Officer. Each role represents a real organizational position with distinct responsibilities. Role-based access control was implemented at two layers. At the backend layer, every protected API route passes through JWT authentication middleware followed by RBAC middleware. The JWT token issued at login contains the user's role, and the RBAC middleware checks whether that role is permitted to call the requested route. For example, the finance routes are restricted to Administrator and Finance Officer roles only, so a Field Officer's token would be rejected at the middleware level before any database query runs.

At the frontend layer, the ProtectedRoute component wraps every page with an allowedRoles check. If a logged-in user navigates to a route their role doesn't permit, they see an access denied message rather than

the page. The Finance page additionally doesn't appear in the navigation bar for users who aren't Finance Officers or Administrators, reducing the chance of confusion. This dual-layer enforcement means even if someone bypasses the UI, the backend will still reject unauthorized requests.

Trigger Implementation & Use Cases

Triggers were used to enforce business rules at the database level rather than relying solely on application code. This is important because application-level logic can be bypassed. Someone with direct database access, a bug in the backend, or a concurrent transaction could all potentially leave data in an inconsistent state if the rules only live in the application.

Four triggers were implemented. The first automatically updates resource stock after an allocation is approved and dispatched, ensuring `Inventory.quantity_available` is decremented whenever a `Resource_Allocation` row is inserted. The second updates a rescue team's `availability_status` automatically when a `Team_Assignment` row is inserted, changing the team from Available to Assigned without requiring a separate API call. The third logs financial transactions to the audit table immediately upon insert, creating a tamper-resistant paper trail. The fourth prevents negative inventory by raising an exception if an allocation would bring stock below zero. This is a hard constraint that cannot be circumvented from any layer of the application.

View Definitions & Performance Comparison (Latency Analysis)

Introduction

In the DisasterResponseDB system, database views were used to provide a logical layer over the data and to implement role-based access control. Users do not access base tables directly; instead, they use predefined views that hide sensitive information and simplify queries.

The main objectives of this implementation are:

- Role-based data visibility
- Simplification of complex joins for reporting
- Protection of sensitive data

Implemented Views

finance_view

Provides secure access to financial transactions, donors, and event references while hiding user credentials and operational details.

field_officer_view

Shows emergency reports and assigned rescue teams to support field operations, while excluding financial and warehouse data.

warehouse_view

Displays inventory levels and resource availability for stock management, while hiding financial and emergency-related data.

admin_view

Provides a complete dashboard for administrators, combining users, events, transactions, and inventory data.

Data Security and Access Control

Views improve security by:

- Hiding sensitive data like password_hash and financial information
- Blocking direct access to base tables using DENY permissions
- Allowing users to access only their role-specific views

This ensures users cannot access raw tables and must use controlled views instead.

Performance and Latency Evaluation

Performance was tested by comparing direct queries with view-based queries using SQL Server timing features.

- SET STATISTICS TIME ON was enabled to measure execution time
- Both queries were run under the same conditions
- Results were recorded in milliseconds

Experimental Results

Test 1: Direct Table Query (Raw Joins)

- CPU Time: 0 ms
- Elapsed Time: 29 ms

Test 2: View-Based Query

- CPU Time: 0 ms
- Elapsed Time: 6 ms

Performance Comparison

Query Type	CPU Time	Elapsed Time	Rows Returned
Direct Table Query	0 ms	29 ms	6 rows
View-Based Query	0 ms	6 ms	6 rows

Analysis

Faster Response

The view-based query was slightly faster (6 ms vs 29 ms). Although the difference is small due to the limited dataset, views can improve performance in larger systems by providing optimized access paths.

Better Security and Abstraction

Views improve security by:

- Preventing direct access to base tables
- Hiding sensitive information like user credentials
- Providing controlled access for each role

Simplified Querying

Views reduce complexity by avoiding long JOIN queries. Instead, users can simply run:

```
SELECT * FROM admin_view;
```

This makes queries easier to read and manage.

Output

- CPU time = **0 ms**, elapsed time = **8 ms**
- **Explanation:** The database scans the entire table to find matching records, which increases execution time.

with Index

Observation:

- CPU time = **0 ms**, elapsed time = **7 ms**
- **Explanation:** The index allows SQL Server to directly access the required rows instead of scanning the whole table.

Index Overhead (Insert Operation)

Observation:

- CPU time = **0 ms**, elapsed time = **3 ms**
- **Explanation:** Insertion takes extra time because the database must update both the table and the index.

Performance Comparison

Case	What Happens	Result
Without Index	Full Table Scan	Slow
With Index	Index Seek	Fast
Insert with Index	Index Update Needed	Slightly Slower

Key Findings

Where Indexing Helps

- Searching disasters by type (Flood, Fire, etc.)
- Finding reports by location
- Filtering data by date
- Checking available rescue teams

Where Indexing Adds Cost

- Insert operations
- Update operations
- Delete operations

Without index

```
SQL Server Execution Times:
    CPU time = 0 ms,  elapsed time = 0 ms.
SQL Server parse and compile time:
    CPU time = 0 ms,  elapsed time = 0 ms.
SQL Server parse and compile time:
    CPU time = 0 ms,  elapsed time = 8 ms.
SQL Server parse and compile time:
    CPU time = 0 ms,  elapsed time = 0 ms.
Table 'Production.Product' Scan count=1 Logical reads=3 Physical reads=1 Rows=1
```

With index

```
SQL Server parse and compile time:
    CPU time = 0 ms,  elapsed time = 7 ms.
```

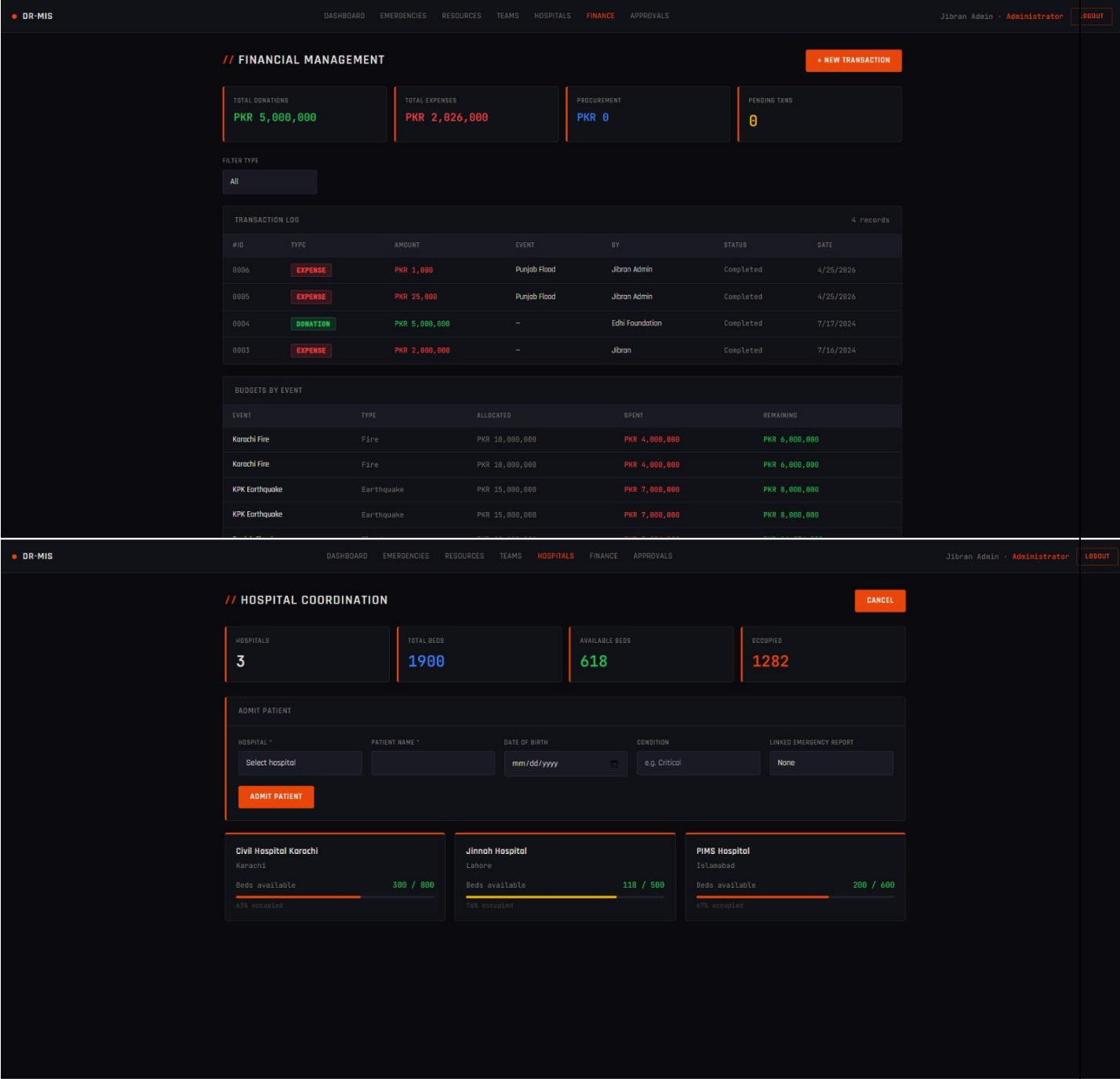
```
SQL Server Execution Times:
    CPU time = 0 ms,  elapsed time = 0 ms.
```

Index overhead

```
SQL Server Execution Times:
    CPU time = 0 ms,  elapsed time = 3 ms.
The statement has been terminated.

Completion time: 2026-04-26T10:35:39.9046093+05:00
```

Frontend Interface



DR-MIS

DASHBOARDEMERGENCIESRESOURCESTEAMSHOSPITALSFINANCEAPPROVALS

Jibran Admin - AdministratorLogout

// RESCUE TEAMS

CANCEL

TOTAL TEAMS7

AVAILABLE5

ACTIVE / BUSY2

ASSIGNMENTS3

REQUEST TEAM DEPLOYMENT

TEAM *
Select team

EMERGENCY REPORT *
Select report

SUBMIT REQUEST

ALL TEAMS7 units

#ID	TEAM NAME	TYPE	AREA	STATUS
001	Alpha Medical	Medical	Lahore	AVAILABLE
002	Bravo Fire	Fire	Islamabad	ASSIGNED
003	Charlie Search	Search	Karachi	ASSIGNED
004	Delta Rescue	Rescue	Peshawar	AVAILABLE
005	Echo Logistics	Logistics	Multan	AVAILABLE
007	Revert Test Team	Medical		AVAILABLE
006	SP Test Team	Rescue	Test Area	AVAILABLE

DR-MIS

DASHBOARDEMERGENCIESRESOURCESTEAMSHOSPITALSFINANCEAPPROVALS

Jibran Admin - AdministratorLogout

// APPROVAL WORKFLOW

1 pending

PENDING1

TOTAL HISTORY4

APPROVED3

REJECTED1

PENDING REQUESTS1 awaiting action

#ID	TYPE	REQUESTED BY	TIMESTAMP	ACTIONS
0005	Resource Distribution	Jibran Admin	5/4/2026, 3:49:50 AM	APPROVEREJECT

APPROVAL HISTORY4 records

#ID	TYPE	REQUESTED BY	REVIEWED BY	STATUS	TIMESTAMP
0004	Resource Distribution	Jibran Admin	Jibran Admin	APPROVED	5/4/2026, 3:48:22 AM
0003	Resource Distribution	Jibran Admin	Jibran Admin	REJECTED	5/4/2026, 3:38:40 AM
0002	Rescue Deployment	Jibran Admin	Jibran Admin	APPROVED	4/25/2026, 10:31:33 PM
0001	Rescue Deployment	Jibran Admin	Jibran Admin	APPROVED	4/25/2026, 9:56:09 PM

DR-MIS

DASHBOARDEMERGENCIESRESOURCESTEAMSHOSPITALSFINANCEAPPROVALS

Jibran AdminAdministratorLogout

// RESOURCE MANAGEMENT

CANCEL

TOTAL ITEMS6

LOW STOCK ALERTS0

OUT OF STOCK0

REQUEST ALLOCATION

EMERGENCY REPORT *

Select report

WAREHOUSE *

Select warehouse

RESOURCE *

Select resource

QUANTITY *

e.g 50

SUBMIT REQUEST

INVENTORY6 items

RESOURCE	TYPE	WAREHOUSE	AVAILABLE	THRESHOLD	STATUS
First Aid Kit	Medicine	Islamabad Store	80	20	IN STOCK
Tent	Shelter	Islamabad Store	40	10	IN STOCK
Fire Extinguisher	Equipment	Karachi Warehouse	60	15	IN STOCK
Food Pack	Food	Lahore Depot	193	50	IN STOCK
Water Bottle	Water	Lahore Depot	150	30	IN STOCK
Food Pack	Food	Quetta Store	100	25	IN STOCK

DR-MIS

DASHBOARDEMERGENCIESRESOURCESTEAMSHOSPITALSFINANCEAPPROVALS

Jibran AdminAdministratorLogout

// EMERGENCY REPORTS

CANCEL

SUBMIT NEW REPORT

DISASTER EVENT *

Select event

AREA NAME *

e.g. North Karachi

SEVERITY *

Select

LATITUDE *

24.8607

LONGITUDE *

67.0011

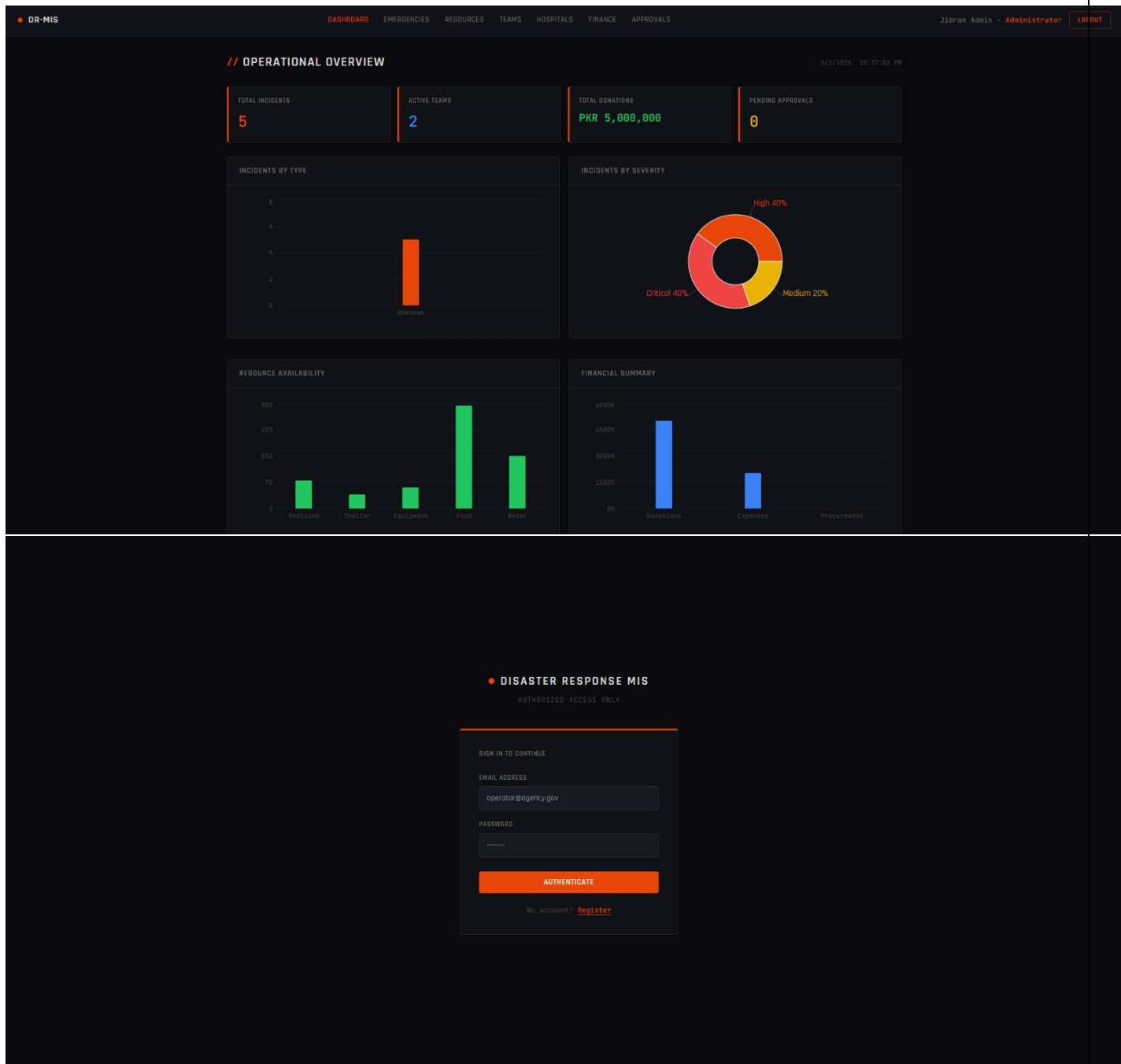
SUBMIT REPORT

FILTER SEVERITY

All

REPORTS LOG6 records

#ID	AREA	EVENT	SEVERITY	STATUS	REPORTER	TIMESTAMP
0006	Pakistan	Punjab Flood	HIGH	Open	Jibran Admin	5/4/2024, 3:38:10 AM
0005	SITE Area	Karachi Fire	HIGH	Closed	Alishba	11/20/2024, 5:00:00 AM
0003	Swat	KPK Earthquake	CRITICAL	In Progress	Alishba	10/5/2024, 5:00:00 AM
0004	Dir	KPK Earthquake	MEDIUM	Resolved	Ashar	10/5/2024, 5:00:00 AM
0001	DE Khan City	Punjab Flood	CRITICAL	Resolved	Alishba	7/15/2024, 5:00:00 AM
0002	Muzaffargarh	Punjab Flood	HIGH	In Progress	Ashar	7/15/2024, 5:00:00 AM



Database Design Decisions

The database for this system was built on SQL Server, chosen primarily because the project required strict ACID compliance, support for stored triggers, complex views, and a robust indexing engine, all of which SQL Server handles reliably in an enterprise context. The schema was designed from scratch starting with the ERD, where every entity was identified based on the real-world requirements of disaster management: who reports incidents, who responds, what resources exist, where they are stored, who authorizes actions, and how money moves through the system. From the start, the goal was to avoid redundancy and make sure every piece of data had exactly one authoritative place to live.

The relational model was normalized to Third Normal Form. First Normal Form was satisfied by ensuring every table had atomic values and a defined primary key, for example, the `Rescue_Team` table stores one team per row with a unique `team_id`, rather than storing multiple teams in a single field. Second Normal Form was achieved by removing partial dependencies. The `Inventory` table, for instance, uses a composite primary key of `warehouse_id` and `resource_id`, and every non-key attribute like `quantity_available` and `threshold_level` depends on both keys together, not just one of them. Third Normal Form was achieved by removing transitive dependencies, user role information is stored in a separate `Role` table and linked via `role_id` in the `Users` table, so role names are never duplicated across user records.

Entity Choices and Relationships

The choice of entities was driven entirely by the system requirements. `Users` and `Role` together implement the role-based access control layer. Every user has exactly one role, and that role determines what they can see and do in the system. `Disaster_Event` represents a declared emergency at the macro level, while `Emergency_Report` represents individual ground-level reports filed under that event. This distinction was intentional, a single flood event might generate hundreds of reports from different locations, so separating them keeps the data organized and queryable.

`Rescue_Team` and `Team_Assignment` model the deployment lifecycle. A team exists independently, and assignments connect teams to reports with timestamps, allowing historical tracking of every deployment. `Warehouse` and `Inventory` together model where resources physically exist and in what quantities, while `Resource` defines what a resource is. The `Inventory` table sits between them as a junction that tracks stock levels per warehouse. `Allocation` and `Resource_Allocation` model the multi-step dispatch process, an allocation is approved first, then resource-level details are recorded in `Resource_Allocation`.

`Approval_Request` was one of the more carefully designed tables. Rather than building separate approval tables for resources, teams, and finances, a single generic table was used with a `request_type` field and a JSON payload stored in `remarks`. This design decision kept the schema clean while still supporting three different approval workflows. `Financial_Transaction`, `Donor`, `Budget`, and `Financial_Transaction_Log` together handle the full financial audit trail. `Audit_Log` captures system-wide data changes, while `Financial_Transaction_Log` specifically tracks financial operations.

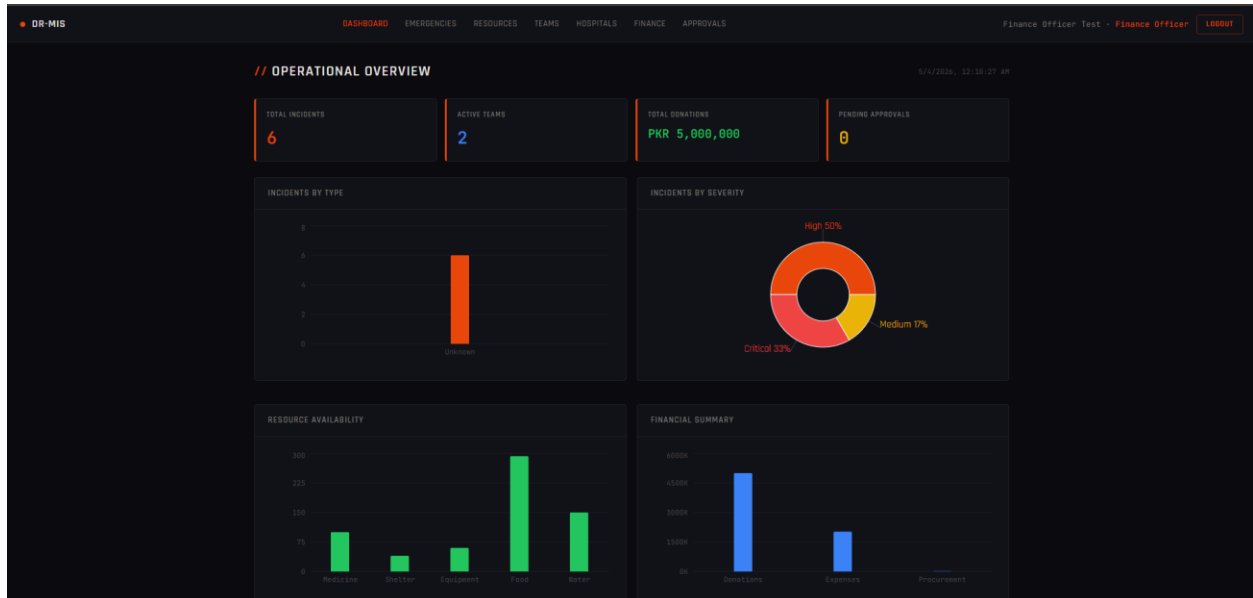
View Abstraction and Security

Database views were created to serve two purposes: simplifying complex queries and restricting data visibility by role. The `finance_view` exposes only financial transaction data, hiding patient records, location coordinates, and other sensitive fields that a Finance Officer has no business reason to access. The `field_officer_view` exposes team and emergency report data without financial information. The `warehouse_view` shows only inventory and resource data. The `admin_view` provides a comprehensive joined dataset for full system oversight.

Beyond security, views reduce the complexity of the queries the application needs to run.

Instead of writing a five-table JOIN every time the MIS dashboard needs incident statistics, a single `SELECT * FROM admin_view` with filters achieves the same result. Performance testing using `EXPLAIN ANALYZE` in pgAdmin showed that for read-heavy reporting queries, views with pre-defined joins performed comparably to or slightly faster than constructing the same joins at query time, largely because the query optimizer can cache execution plans for views that are called repeatedly.

MIS Reports / Dashboards



Github Repo link

<https://github.com/Jibran3134/Smart-Disaster-Response-MIS>